Lab 11 — SOC automation with a human still in the loop

Course: Maxim Nyansa Cybersecurity Bootcamp, Ghana
Trainee: James Ackah Blay · Aiyinasi, Ghana
Week: 11 — AI and cybersecurity automations
Estimated time: 6 hours
Difficulty: Advanced. Scripts count and summarise. You decide.

> **Tutor note.** The compliance exercise also uses the GRC Lab at https://grclab.purplecop.io/lab . Do not paste personal data into a public model. The Wazuh API example uses a token you type from your own install. The tests use a saved JSON fixture, not your server. This lab does not automate offensive tests.

1. Learning objectives

Objective	Job skill
Count a control sheet without calling it an audit	GRC automation and its limit
Triage alerts from a manager API, or from a saved export	SOC analyst workflow at the start of a shift
Map a detection to ATT&CK	Detection engineering vocabulary
Summarise indicators you were given, then stop	Threat intelligence handling
Review a model draft	Human-in-the-loop, which is the point of the week

2. Prerequisites and setup

Python environment from the repository `requirements.txt`. Wazuh from Week 3 if you will call the API. Otherwise use the fixture.

```
python3 -m pytest week-11-ai-soc-automation/compliance-automation
python3 -m pytest week-11-ai-soc-automation/soc-alert-triage
python3 -m pytest week-11-ai-soc-automation/sigma-rules
python3 -m pytest week-11-ai-soc-automation/threat-intel-summary
python3 -m pytest week-11-ai-soc-automation/lab-11-soc-automation
```

3. Scenario

Monday's queue has a compliance spreadsheet, a handful of authentication alerts, and a short indicator list. A teammate wants a chatbot to "close the tickets". You will automate the counting and the first sort, and you will keep the decision.

4. Tasks

Task 1 — Compliance count

```
python3 ../compliance-automation/compliance_report.py ../compliance-automation/sample-controls.csv --markdown
```

The sample is synthetic: 2 met, 2 partial, 2 gap, 2 not-reviewed. Copy the CSV, replace rows with your Week 4 statuses, and run it again. Keep `data_label` honest.

Do the same exercise in the GRC Lab and save the export here. The script does not know what you clicked there.

Task 2 — Alerts, offline first

```
python3 wazuh_offline.py fixtures/alerts.json
```

> **You should now see** two alerts, one brute-force and one new admin, each with a technique id, and a line that says a person must confirm before you close anything. The fixture is labelled synthetic.

Task 3 — Live API, if the manager is up

On the Ubuntu VM, use the API password from your install notes. Do not paste it into this repository.

```
        TOKEN=$(curl -sk -u "wazuh-wui:YOUR_PASSWORD" -X POST "https://127.0.0.1:55000/security/user/authenticate?raw=true")
        curl -sk -H "Authorization: Bearer $TOKEN" "https://127.0.0.1:55000/manager/info?pretty=true"
```

Wazuh 4.x uses port 55000 for the manager API. If the path differs, use the API reference for the version you installed and write the path that worked. Save `manager/info` only. Do not save the token.

If you export alerts, strip usernames that are real people before the file is committed. Prefer to keep the export out of git and describe it in your notes.

Task 4 — Sigma and ATT&CK

```
        python3 ../sigma-rules/sigma_check.py ../sigma-rules/rules/*.yml
```

In your notes, map the lab rules: failed SSH and Windows 4625 to T1110, and a new sudo or Administrators member to T1098. Say which rule only selects the event, and which threshold lives in `log_triage.py` (five failures in ten minutes).

Task 5 — Indicators and a model

```
        python3 ../threat-intel-summary/threat_intel_summary.py ../threat-intel-summary/sample-feed.json
```

Answer the human-review questions in `review-notes.md`. If you ask a model to draft a sentence, give it the counts and the technique ids, not raw customer data. Then edit the sentence. The draft is not the submission until you have changed what was wrong.

5. Troubleshooting

What you see	What it means
API connection refused	The manager is down, or you are not on the VM
HTTP 401	Wrong user or password. Rotate it if you pasted it into a chat
`sigma_check` fails a rule	A required field or the `attack.t` tag is missing
The offline script and the dashboard disagree	The fixture is not your server. Say which one you are writing about

6. Stretch task

Add one alert to a copy of `fixtures/alerts.json` that should not be auto-closed, such as a single failed logon. Show that your notes still require a person. Do not lower the human-review line to make the queue look empty.

7. Evidence

- Markdown count of your control CSV, or the sample clearly labelled.
- Offline triage output.
- Optional `manager/info` with the token removed.
- Sigma check output.
- `review-notes.md` with the model draft and your edit, or “no model used”.

8. Reflection

1. What did the script decide, and what did you decide?
2. Which ATT&CK technique matches the new admin, and what does that change in the response?
3. What data must never enter a public model?

9. Interview questions

1. Why is a control count not an audit opinion?
2. How do you triage an alert before you call it an incident?
3. What does a Sigma rule buy you if you already have a script?
4. Where does a human stay in the loop when a model drafts the ticket?

10. Grading rubric

Criterion	Exceeds	Meets	Not yet
Compliance	Your Week 4 sheet is counted and labelled	Sample count explained	Sample presented as your audit
Alerts	Live export, redacted, compared with the fixture	Fixture triage is correct	Token committed
Detection mapping	You can say where the threshold lives	Four rules pass `sigma_check`	Technique tags removed to “make it simpler”
Judgement	A model draft is corrected in your words	Review notes exist	Unedited model text submitted as yours